

Question on Bigram and perplexity:

Formal Definition

For a sentence with probabilities $P_1, P_2, \dots, P_N$:

$$\text{Perplexity} = \left(\frac{1}{P_1 \cdot P_2 \cdot \dots \cdot P_N} \right)^{1/N}$$

Or more commonly in log form:

$$\text{Perplexity} = e^{\text{Cross-Entropy}}$$

Q1.

Given a sentence:

“I love NLP”

Using chain rule, expand the probability: $P(I, \text{love}, \text{NLP})$

Expected Answer:

$$P(I) \cdot P(\text{love} | I) \cdot P(\text{NLP} | I, \text{love})$$

(Compute Probability)

Given:

- $P(I) = 0.1$
- $P(\text{love} | I) = 0.5$
- $P(\text{NLP} | I, \text{love}) = 0.2$

Find: $P(I, \text{love}, \text{NLP})$

Solution:

$$0.1 \times 0.5 \times 0.2 = 0.01$$

2. Bigram Language Model

Assume a bigram model (only previous word matters):

$$P(w_n | w_1, w_2, \dots, w_{n-1}) \approx P(w_n | w_{n-1})$$

Rewrite:

$$P(I, \text{love}, \text{NLP})$$

Answer:

$$P(I) \cdot P(\text{love} | I) \cdot P(\text{NLP} | \text{love})$$

(Numerical Bigram)

Given:

- $P(I) = 0.1$
- $P(\text{love} | I) = 0.5$
- $P(\text{NLP} | \text{love}) = 0.4$

Compute sentence probability.

Solution:

$$0.1 \times 0.5 \times 0.4 = 0.02$$

Compute perplexity for a sentence of 3 words with probabilities:

$$P = 0.1, 0.5, 0.2$$

Use formula:

$$\text{Perplexity} = \left(\frac{1}{P_1 \cdot P_2 \cdot P_3} \right)^{1/3}$$

Solution:

$$= \left(\frac{1}{0.1 \times 0.5 \times 0.2} \right)^{1/3} = (100)^{1/3} \approx 4.64$$

2. Parameter Count

Question

You have a transformer model with a linear layer

$$W \in \mathbb{R}^{4096 \times 4096}.$$

You want to apply LoRA (Low-Rank Adaptation) with:

- Rank: $r = 16$
- LoRA matrices:

$$A \in \mathbb{R}^{r \times d_{\text{in}}}$$

$$B \in \mathbb{R}^{d_{\text{out}} \times r}$$

Assume the original weight matrix is frozen and only the LoRA parameters are trainable.

1. How many trainable parameters are added by LoRA for this layer?
2. Compare this with full fine-tuning of the original layer.
3. What percentage reduction in trainable parameters does LoRA achieve?

Solution

In LoRA, the weight update is represented as:

$$\Delta W = BA$$

where:

$$A \in \mathbb{R}^{r \times d_{\text{in}}}$$

and

$$B \in \mathbb{R}^{d_{\text{out}} \times r}.$$

The total number of trainable parameters is therefore:

$$r \times d_{\text{in}} + d_{\text{out}} \times r$$

we get:

$$d_{\text{in}} = 4096$$

$$d_{\text{out}} = 4096$$

$$r = 16$$

$$\text{LoRA parameters} = 16 \times 4096 + 4096 \times 16$$

$$= 65536 + 65536$$

$$= 131072$$

Answer to Part 1

131072 trainable parameters

Part 2: Full Fine-Tuning Parameters

The original weight matrix contains:

$$4096 \times 4096 = 16777216$$

parameters.

Thus, full fine-tuning requires:

16777216 trainable parameters

Part 3: Percentage Reduction

The reduction percentage is:

$$\begin{aligned}\text{Reduction} &= \left(1 - \frac{131072}{16777216}\right) \times 100 \\ &= (1 - 0.0078125) \times 100 \\ &= 99.21875\%\end{aligned}$$

Question: Parameter Count in Multi-Head Attention

Consider a Transformer Multi-Head Attention (MHA) layer with:

- Embedding dimension: $d_{\text{model}} = 768$
- Number of attention heads: $h = 12$

Assume the layer contains the following trainable projection matrices:

$$W_Q, W_K, W_V, W_O$$

where each matrix has dimensions:

$$768 \times 768$$

Ignore bias parameters.

1. Find the total number of trainable parameters in the Multi-Head Attention layer.
2. If the model instead used a single-head attention mechanism with the same embedding dimension, would the parameter count change?
3. Explain why or why not.

Solution

The Multi-Head Attention layer contains four projection matrices:

$$W_Q, W_K, W_V, W_O$$

Each matrix has dimensions:

$$768 \times 768$$

Therefore, parameters per matrix:

$$768 \times 768 = 589824$$

Since there are four such matrices:

$$4 \times 589824 = 2359296$$

Part 2: Single-Head Attention

Even with single-head attention, the projection matrices are still:

$$768 \times 768$$

for:

$$W_Q, W_K, W_V, W_O$$

Thus, the total parameter count remains:

$$\boxed{2359296}$$

Part 3: Explanation

The number of heads changes how the embeddings are split internally during attention computation, but the overall projection matrices still map:

$$d_{\text{model}} \rightarrow d_{\text{model}}$$

Therefore, the total number of trainable parameters does not depend on the number of attention heads when the embedding dimension is fixed.